

PRAKTIKUM MACHINE LEARNING

Bab 2

Workflow Machine Learning, CRISP-DM, dan Diagnosis Bias-Variance

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

2 Oktober 2026

Daftar Isi

Workflow Machine Learning, CRISP-DM, dan Diagnosis Bias-Variance	2
Ringkasan Konsep	3
2.13 Latihan Praktikum (Modul Bab 2)	5
Percobaan Mandiri	11
Kesimpulan Bab 2	13

Tools yang Digunakan

Python 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

Praktikum Machine Learning: Bab 2

Workflow Machine Learning, CRISP-DM, dan Diagnosis Bias-Variance

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Notebook ini saya kerjakan di Google Colab / Jupyter Notebook.

Ringkasan Konsep

- **CRISP-DM**: kerangka baku proyek ML dengan 6 fase iteratif: 1) Business Understanding, 2) Data Understanding, 3) Data Preparation, 4) Modeling, 5) Evaluation, 6) Deployment.
- **Train/validation/test split**: train melatih model, validation memilih dan men-tuning model, test menilai hasil akhir tanpa pernah dipakai memilih model.
- **Stratified sampling**: pembagian data yang menjaga proporsi tiap kelas di setiap belahan, penting untuk dataset yang tidak seimbang.
- **Bias-variance tradeoff**: bias tinggi berarti model terlalu sederhana (underfitting), variance tinggi berarti model menghafal noise data latih (overfitting). Model ideal menyeimbangkan keduanya.
- **Learning curve**: plot skor train vs validasi terhadap jumlah data latih. Rendah dan rapat berarti underfitting, gap besar berarti overfitting, tinggi dan rapat berarti seimbang.

Sel 0 - Persiapan Awal (Khusus Google Colab)

Latihan 2 bab ini memakai **data sintetis** `make_classification` yang dibuat langsung di cell latihan, jadi tidak butuh unduhan. Satu-satunya file yang dibutuhkan bab ini adalah dataset Telco Customer Churn untuk Latihan 1 (CRISP-DM). Kalau notebook dibuka di Google Colab lewat link GitHub, jalankan sel di bawah ini dulu. Kalau dijalankan di laptop dan file sudah ada, sel ini langsung dilewati.

In [1] - Kode Program

```
1 # Sel 0 - Persiapan awal (jalan otomatis paling awal saat Run All).
2 # Mengunduh file data dari repo GitHub kalau belum ada (kasus dibuka di Google
  Colab).
3 # Di laptop (folder data/ sudah ada), bagian unduh langsung dilewati.
4 import os
5 import urllib.request
6
7 _BAB = "bab-02-workflow-crisp-dm-bias-variance" # nama folder bab ini di repo
  GitHub
8 _REPO = "https://raw.githubusercontent.com/dandienter/praktikumML/main"
9 _DATA = ["data/telco-customer-churn.csv"] # hanya untuk cuplikan Latihan 1
10
11 for _f in _DATA:
12     if os.path.exists(_f):
13         print(f"OK: {_f} sudah ada.")
14         continue
15     os.makedirs(os.path.dirname(_f), exist_ok=True)
16     print(f"mengunduh {_f} dari GitHub ...")
17     urllib.request.urlretrieve(f"{_REPO}/{_BAB}/{_f}", _f)
18 print("Data siap, silakan lanjutkan.")
```

OK: data/telco-customer-churn.csv sudah ada.
Data siap, silakan lanjutkan.

2.13 Latihan Praktikum (Modul Bab 2)

Latihan 1 - Memetakan Proyek Nyata ke CRISP-DM (Modul Bab 2, bagian 2.13)

Soal: pilih studi kasus **prediksi churn pelanggan telekomunikasi** (dataset Telco Customer Churn). Tuliskan aktivitas konkret di 6 fase CRISP-DM.

Supaya pemetaannya konkret (bukan karangan), saya intip dulu datasetnya: `data/telco-customer-churn.csv`.

In [2] - Kode Program

```
1 # Latihan 1 - impor paket yang dibutuhkan (mandiri, tidak bergantung cell lain)
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 # Cuplikan data Telco Customer Churn untuk Latihan 1
7 churn = pd.read_csv("data/telco-customer-churn.csv")
8 print("Shape:", churn.shape)
9 print("\nDistribusi Churn (target):")
10 print(churn['Churn'].value_counts())
11 print((churn['Churn'].value_counts(normalize=True) * 100).round(2))
12 kosong = (churn['TotalCharges'].astype(str).strip() == '').sum()
13 print("\nBaris dengan TotalCharges kosong:", kosong)
14 print("\nRata-rata tenure (bulan) per status churn:")
15 print(churn.groupby('Churn')['tenure'].mean().round(1))
16 print("\nRata-rata MonthlyCharges per status churn:")
17 print(churn.groupby('Churn')['MonthlyCharges'].mean().round(1))
18 print("\nChurn rate per jenis kontrak:")
19 print(pd.crosstab(churn['Contract'], churn['Churn'], normalize='index').round(3))
```

Shape: (7043, 21)

Distribusi Churn (target):

Churn

No 5174

Yes 1869

Name: count, dtype: int64

Churn

No 73.46

Yes 26.54

Name: proportion, dtype: float64

Baris dengan TotalCharges kosong: 11

Rata-rata tenure (bulan) per status churn:

Churn

No 37.6

Yes 18.0

Name: tenure, dtype: float64

Rata-rata MonthlyCharges per status churn:

Churn

No 61.3

Yes 74.4

Name: MonthlyCharges, dtype: float64

Churn rate per jenis kontrak:

Churn	No	Yes
-------	----	-----

Contract

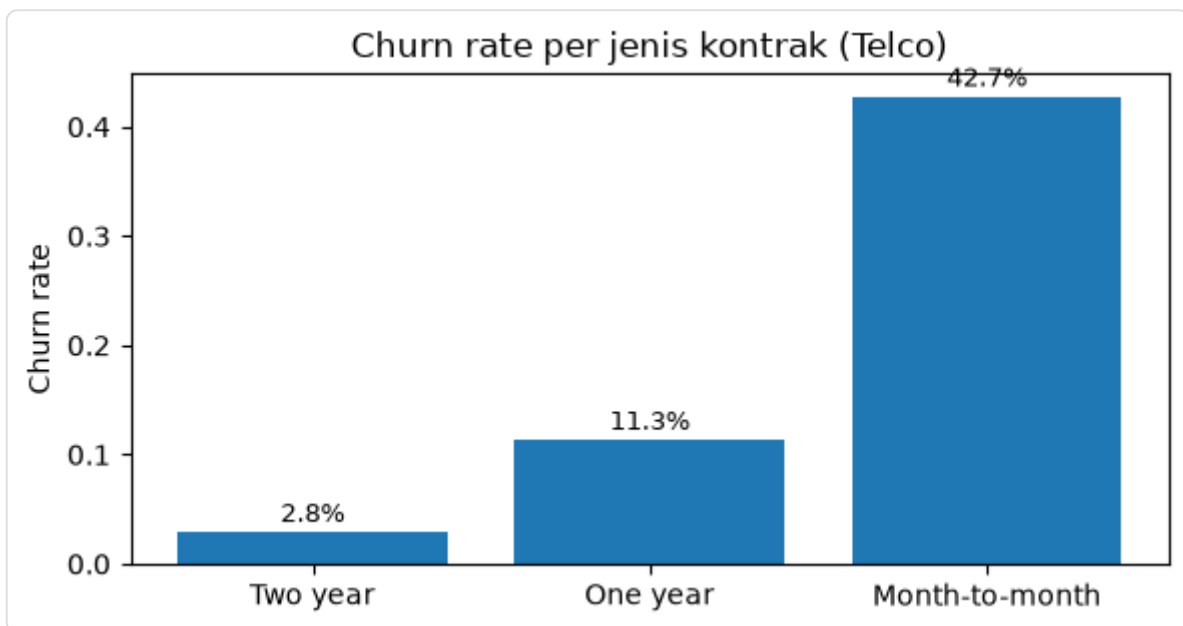
Month-to-month	0.573	0.427
----------------	-------	-------

One year	0.887	0.113
----------	-------	-------

Two year	0.972	0.028
----------	-------	-------

In [3] - Kode Program

```
1 # Grafik: churn rate per jenis kontrak (sinyal bisnis terkuat di Latihan 1)
2 ct = pd.crosstab(churn['Contract'], churn['Churn'], normalize='index')
3 ct.sort_values()
4 plt.figure(figsize=(6, 3.2))
5 plt.bar(ct.index, ct.values)
6 plt.ylabel("Churn rate")
7 plt.title("Churn rate per jenis kontrak (Telco)")
8 for i, v in enumerate(ct.values):
9     plt.text(i, v + 0.01, f"{v:.1%}", ha="center", fontsize=9)
10 plt.tight_layout()
11 plt.show()
```



Gambar 1

Penjelasan grafiknya: pelanggan kontrak **month-to-month** churn paling tinggi (sekitar 42,7%), sedangkan kontrak 1 tahun dan 2 tahun jauh lebih setia (11,3% dan 2,8%). Ini sinyal bisnis paling kuat: model churn harus memperhatikan Contract, dan tim retensi bisa menargetkan pelanggan bulanan dengan penawaran kontrak tahunan.

Penjelasan outputnya: - Dataset: **7.043 pelanggan, 21 kolom**, targetnya Churn (Yes/No) dengan churn **1.869 pelanggan (26,54%)**: tidak seimbang, evaluasi tidak boleh mengandalkan akurasi mentah. - Ada **11 baris** dengan TotalCharges kosong (terbaca string kosong, bukan NaN): data kotor yang harus dibersihkan di fase Data Preparation. - Sinyal bisnis: pelanggan churn rata-rata baru **18,0 bulan** berlangganan (vs 37,6 yang bertahan) dengan tagihan bulanan **lebih mahal** (74,4 vs 61,3). Kontrak month-to-month churn **42,7%**, kontrak 2 tahun cuma **2,8%**.

Jawaban: pemetaan ke 6 fase CRISP-DM:

Fase	Aktivitas konkret pada proyek churn
1. Business Understanding	Tujuan: menurunkan churn. Sukses bila model mengidentifikasi 80% calon churn dengan presisi yang bisa ditindaklanjuti tim retensi. Metrik utama: recall kelas churn (lebih baik salah alarm daripada kehilangan pelanggan).
2. Data Understanding	Eksplorasi 7.043 baris x 21 kolom: churn 26,54% (imbalance), 11 baris TotalCharges kosong, fitur kandidat kuat: Contract (month-to-month churn 42,7%), tenure (18,0 vs 37,6 bulan), MonthlyCharges (74,4 vs 61,3).
3. Data Preparation	Konversi TotalCharges ke numerik (11 baris kosong jadi NaN lalu imputasi median), one-hot encoding kolom kategorikal, scaling fitur numerik, stratified split 80/20 menjaga proporsi churn 26,54%.
4. Modeling	Baseline Logistic Regression, lalu Decision Tree dan Random Forest; tuning lewat learning curve + validasi silang ber-stratify; bandingkan F1/recall kelas churn, bukan sekadar akurasi.
5. Evaluation	Uji di test set yang belum tersentuh: confusion matrix dan precision-recall; validasi dengan tim bisnis, apakah daftar "pelanggan berisiko" masuk akal (misalnya didominasi kontrak bulanan + tenure pendek)? Kalau tidak, kembali ke fase 2/3.
6. Deployment	Deploy sebagai skor churn mingguan di dashboard CRM; skor tinggi otomatis masuk prioritas tim retensi (tawaran kontrak tahunan/diskon); monitor data drift, latih ulang kalau pola churn berubah.

Latihan 2 - Diagnosis Bias-Variance (Praktikum 2.7) (Modul Bab 2, bagian 2.13)

Soal: bandingkan learning curve untuk max_depth berbeda, lalu tulis diagnosis (underfitting/overfitting/seimbang) untuk setiap nilai.

In [4] - Kode Program

```
1 # Latihan 2 - data dan paket dibuat di sini (mandiri, tidak bergantung cell lain)
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split, learning_curve
4 from sklearn.tree import DecisionTreeClassifier
5
6 X, y = make_classification(n_samples=1000, n_features=10,
7     n_informative=6, weights=[0.8, 0.2], random_state=42)
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.2, random_state=42, stratify=y)
10 print("Distribusi kelas:", np.bincount(y))
11 print("Proporsi kelas train:", (np.bincount(y_train) / len(y_train)).round(3))
12
13 # Bandingkan learning curve untuk max_depth berbeda
14 for depth in [1, 4, None]:
15     model = DecisionTreeClassifier(max_depth=depth, random_state=42)
16     train_sizes, train_scores, val_scores = learning_curve(model, X_train,
17         y_train,
18         cv=5, train_sizes=np.linspace(0.1, 1.0, 6))
19     print(f"max_depth={depth}: train={train_scores.mean():.3f},
20         val={val_scores.mean():.3f}")
21 # Tuliskan diagnosis (underfitting/overfitting/seimbang) untuk setiap nilai
22 max_depth.
```

Distribusi kelas: [797 203]
Proporsi kelas train: [0.798 0.202]
max_depth=1: train=0.849, val=0.800

max_depth=4: train=0.952, val=0.866

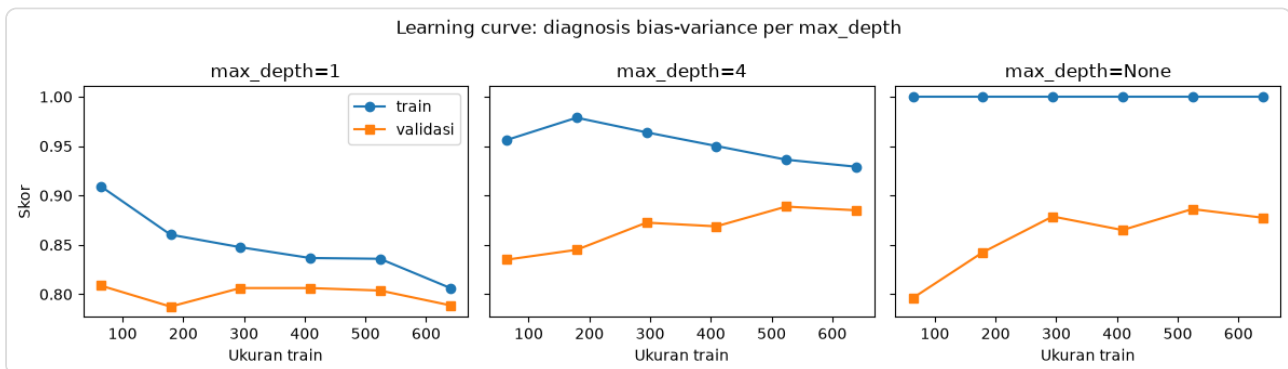
max_depth=None: train=1.000, val=0.858

In [5] - Kode Program

```

1 # Grafik learning curve untuk tiap max_depth (visualisasi diagnosis bias-variance)
2 fig, axes = plt.subplots(1, 3, figsize=(12, 3.4), sharey=True)
3 for ax, depth in zip(axes, [1, 4, None]):
4     m = DecisionTreeClassifier(max_depth=depth, random_state=42)
5     ts, tr, va = learning_curve(m, X_train, y_train, cv=5,
6                               train_sizes=np.linspace(0.1, 1.0, 6))
7     ax.plot(ts, tr.mean(axis=1), 'o-', label="train")
8     ax.plot(ts, va.mean(axis=1), 's-', label="validasi")
9     ax.set_title(f"max_depth={depth}")
10    ax.set_xlabel("Ukuran train")
11 axes[0].set_ylabel("Skor")
12 axes[0].legend()
13 fig.suptitle("Learning curve: diagnosis bias-variance per max_depth")
14 plt.tight_layout()
15 plt.show()

```



Gambar 2

Penjelasan grafiknya: pada `max_depth=1` kedua kurva datar dan rendah (underfitting, bias tinggi); pada `max_depth=4` kedua kurva tinggi dan gap-nya wajar (seimbang); pada `max_depth=None` kurva train menempel di 1,000 sementara validasi tertinggal jauh di bawah (overfitting, variance tinggi).

Jawaban: diagnosis tertulis tiap nilai max_depth: - **max_depth=1 (underfitting):** skor train 0.849 dan validasi 0.800 sama-sama rendah dengan gap kecil, pohon sedalam 1 lapis terlalu sederhana untuk menangkap pola data (bias tinggi). - **max_depth=4 (seimbang):** skor train 0.952 dan validasi 0.866 keduanya tinggi dengan gap wajar, jadi ini konfigurasi paling seimbang di antara ketiganya. - **max_depth=None (overfitting):** skor train 1.000 tapi validasi cuma 0.858 dengan gap 0.142, pohon tumbuh bebas sampai menghafal data latih (variance tinggi).

Percobaan Mandiri

Tiga percobaan tambahan yang saya lakukan sendiri di data sintetis, tidak tumpang tindih dengan latihan.

Percobaan Mandiri 1 - Split tanpa stratify vs dengan stratify

Saya bandingkan proporsi kelas di train/test kalau split dilakukan tanpa stratify versus dengan stratify=y.

In [6] - Kode Program

```
1 # Setup data Percobaan Mandiri (mandiri, tidak bergantung cell lain)
2 import numpy as np
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5
6 X, y = make_classification(n_samples=1000, n_features=10,
7     n_informative=6, weights=[0.8, 0.2], random_state=42)
8
9 # Split TANPA stratify
10 Xtr_ns, Xte_ns, ytr_ns, yte_ns = train_test_split(X, y, test_size=0.2,
11     random_state=42)
12 # Split DENGAN stratify
13 Xtr_s, Xte_s, ytr_s, yte_s = train_test_split(X, y, test_size=0.2,
14     random_state=42, stratify=y)
15 print("Tanpa stratify - train:", np.round(np.bincount(ytr_ns)/len(ytr_ns), 4),
16     "| test:", np.round(np.bincount(yte_ns)/len(yte_ns), 4))
17 print("Dengan stratify - train:", np.round(np.bincount(ytr_s)/len(ytr_s), 4),
18     "| test:", np.round(np.bincount(yte_s)/len(yte_s), 4))
```

```
Tanpa stratify - train: [0.7887 0.2112] | test: [0.83 0.17]
Dengan stratify - train: [0.7975 0.2025] | test: [0.795 0.205]
```

Kesimpulan: tanpa stratify proporsi kelas 1 bergeser cukup jauh (train 21,1% vs test 17,0%), sedangkan dengan stratify proporsi terjaga (train 20,3% vs test 20,5%). Jadi stratify menjaga distribusi kelas tetap mewakili aslinya, penting supaya evaluasi di test set tidak bias karena kebetulan pembagian.

Percobaan Mandiri 2 - Decision Tree vs Logistic Regression

Saya bandingkan akurasi test antara Decision Tree (max_depth=4) dan Logistic Regression pada data sintetis yang sama.

In [7] - Kode Program

```
1 # Setup data Percobaan Mandiri (mandiri, tidak bergantung cell lain)
2 import numpy as np
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5
6 X, y = make_classification(n_samples=1000, n_features=10,
7     n_informative=6, weights=[0.8, 0.2], random_state=42)
8 from sklearn.tree import DecisionTreeClassifier
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.metrics import accuracy_score
11
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.2, random_state=42, stratify=y)
14 dt = DecisionTreeClassifier(max_depth=4, random_state=42).fit(X_train, y_train)
15 lr = LogisticRegression(max_iter=2000).fit(X_train, y_train)
16 print("Akurasi test DecisionTree      :", round(accuracy_score(y_test,
17 dt.predict(X_test)), 3))
18 print("Akurasi test LogisticRegression:", round(accuracy_score(y_test,
19 lr.predict(X_test)), 3))
```

```
Akurasi test DecisionTree      : 0.865
Akurasi test LogisticRegression: 0.84
```

Kesimpulan: Decision Tree (0.865) mengalahkan Logistic Regression (0.84) di data ini, wajar karena pola make_classification bersifat non-linear sehingga batas keputusan lurus dari regresi logistik kurang cocok.

Percobaan Mandiri 3 - Logistic Regression dengan `class_weight="balanced"`

Dataset ini imbalance (kelas 1 cuma 20%), jadi saya bandingkan F1 kelas minoritas antara Logistic Regression default dan yang memakai `class_weight="balanced"`.

In [8] - Kode Program

```
1 # Setup data Percobaan Mandiri (mandiri, tidak bergantung cell lain)
2 import numpy as np
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5
6 X, y = make_classification(n_samples=1000, n_features=10,
7     n_informative=6, weights=[0.8, 0.2], random_state=42)
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.metrics import f1_score
10
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y, test_size=0.2, random_state=42, stratify=y)
13 lr = LogisticRegression(max_iter=2000).fit(X_train, y_train)
14 lr_bal = LogisticRegression(max_iter=2000, class_weight="balanced").fit(X_train,
15 y_train)
16 print("F1 kelas minoritas, LogisticRegression default :",
17     round(f1_score(y_test, lr.predict(X_test)), 3))
18 print("F1 kelas minoritas, LogisticRegression balanced :",
19     round(f1_score(y_test, lr_bal.predict(X_test)), 3))
```

```
F1 kelas minoritas, LogisticRegression default : 0.543
F1 kelas minoritas, LogisticRegression balanced : 0.569
```

Kesimpulan: memberi `class_weight="balanced"` menaikkan F1 kelas minoritas dari 0.543 ke 0.569, jadi model sedikit lebih adil dalam mengenali kelas yang jarang muncul tanpa mengorbankan model secara keseluruhan.

Kesimpulan Bab 2

- Pemetaan proyek churn Telco ke 6 fase CRISP-DM menunjukkan tiap fase punya aktivitas konkret: dari target bisnis menurunkan churn, eksplorasi 7.043 pelanggan dengan churn 26,54%, sampai evaluasi memakai recall kelas churn.
- Eksplorasi data menemukan sinyal bisnis terkuat di Contract: pelanggan month-to-month churn 42,7% sedangkan kontrak 2 tahun cuma 2,8%, jadi fitur kontrak wajib diperhatikan model dan tim retensi.
- Diagnosis bias-variance lewat learning curve: `max_depth=1` underfitting (train 0.849, validasi 0.800), `max_depth=4` paling seimbang (train 0.952, validasi 0.866), dan `max_depth=None` overfitting (train 1.000, validasi 0.858).
- Learning curve terbukti jadi alat diagnosis yang praktis: bentuk dan gap kedua kurva langsung menunjukkan apakah model perlu disederhanakan, ditambah datanya, atau justru sudah pas.